

Android Data 관리



목차

 Android의 데이터 저장 옵션

 파일 저장소의 사용

 공유 기본 설정 (Shared Preferences) 의 사용

Android의 데이터 저장 옵션

1. 앱 개별 전용저장소 <-- 파일

- ◆ 앱 전용으로 지정된 내부저장소와 외부저장소
- ◆ 내부저장소 위치 확인: context.filesDir, context.cacheDir
- ◆ 외부저장소 위치 확인: context.getExternalFilesDir(String?), context.getExternalCacheDir
- ◆ 별도 권한은 필요 없으며, 앱 삭제 시 함께 삭제됨

카메라로 찍은 사진을 사진 앱에서 확인 가능

2. 공유 저장소 찍은 사진을 카톡에서 볼 수 있음

- ◆ 미디어, 문서, 기타 파일 등 다른 앱과 공유
- ◆ READ_EXTERNAL_STORAGE, WRITE_EXTERNAL_STORAGE 권한

3. 환경설정 정보

- ◆ 앱 개별 데이터를 key-value 로 앱 전용 내부저장소에 보관
- ◆ SharedPreferences, Preference

4. 데이터베이스

- ◆ 컬럼 3개 이상이 필요한 데이터일 때 고려

파일 저장소의 사용

내부저장소 vs. 외부저장소

특성 \ 분류	내부저장소 파일	외부저장소 파일
종류	앱 전용 파일 or 캐시 파일	앱 지정 파일 or 공용 파일
사용가능성	항상 사용 가능	저장소 분리 등에 의해 사용불가 할 수 있으므로 확인 필요
접근성	파일을 생성한 앱에서만 가능 (전용)	앱 지정 파일도 외부 접근이 가능할 수 있음, 다른 앱에 의해서 접근 가능 (공용)
파일 유지	앱 삭제 시 함께 삭제됨	앱 삭제 시 별도로 지정한 경로가 아닐 경우 파일은 남아 있음 (단, <code>getExternalFilesDir()</code> 에 위치한 디렉토리의 앱 파일은 같이 삭제)
용도	앱 내부에서 자체적으로 사용하는 파일	다른 앱들과 공유할 필요가 있는 파일, 파일 크기 등의 이유로 외부의 전용 공간에 저장할 필요가 있는 파일
주요 접근 메소드	<code>getFilesDir()</code> , <code>getCacheDir()</code>	<code>getExternalFilesDir()</code> , <code>getExternalCacheDir()</code>
기타 사항	퍼미션 불필요 context 로 접근 가능	안드로이드 버전에 따라 퍼미션 추가 필요 (최소 버전 4.4 이상일 경우 불필요)

내부저장소 파일 사용 1

앱 전용의 파일 사용 시 적용

사용 전 용량 확인이 필요할 경우

- ◆ File.freeSpace 로 확인 (File로 사용경로를 지정)

앱 전용 위치

- ◆ 시스템에서 지정하는 위치 → 앱 설치 위치
- ◆ 안드로이드 스튜디오 [Device Explorer]를 사용하여 확인
 - [View] → [Tool Windows] → [Device Explorer]
- ◆ 내부 저장 위치 (바로가기 정보)
 - /data/data/**package_name**/files (또는 cache)
- ◆ 외부 저장 위치 (바로가기 정보)
 - /sdcard/Android/data/**package_name**/files (또는 cache)

경로는 하드코딩이 아닌 실행 시 함수를 사용하여 확인

내부저장소 파일 사용 2

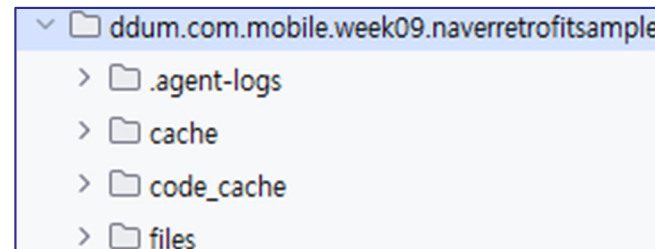
앱 전용 위치의 실제 경로의 확인

- ◆ Context 의 멤버를 사용하여 확인 가능

```
// 내부저장소
Log.d(TAG, "Internal filesDir: ${filesDir}")
Log.d(TAG, "Internal cacheDir: ${cacheDir}")
// 외부저장소
Log.d(TAG, "External filesDir: ${getExternalFilesDir(null).toString()}")
Log.d(TAG, "External cacheDir: ${externalCacheDir}")
```

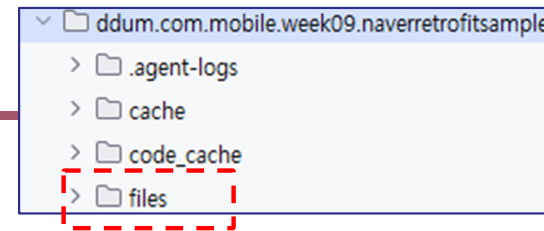


```
Internal filesDir: /data/user/0/ddum.com.mobile.week09.naverretrofitsample/files
Internal cacheDir: /data/user/0/ddum.com.mobile.week09.naverretrofitsample/cache
External filesDir: /storage/emulated/0/Android/data/ddum.com.mobile.week09.naverretrofitsample/files
External cacheDir: /storage/emulated/0/Android/data/ddum.com.mobile.week09.naverretrofitsample/cache
```



• 안드로이드스튜디오 [Device Explorer] 에서 확인 가능

파일 쓰기



```
val writeData = "Mobile Application!"
```

// 기본 방법 - 전용 위치가 아닌 지정 위치 사용

```
val writeFile = File ( filesDir, "file_name")
val outputStream = FileOutputStream(writeFile)
outputStream.write(writeData.toByteArray())
outputStream.close()
```

// Files.write() 대체 사용 가능

• filesDir 이외의 위치도 사용 가능

• true or false 매개변수를 추가하여 파일 덮어 쓰기/추가 지정

• String 을 Byte 배열로 변환하여 OutputStream 에 전달

// 추가 방법 - 앱에 지정되어 있는 전용 위치를 사용

```
context.openFileOutput("file_name", Context.MODE_PRIVATE).use {
    it.write(writeData.toByteArray())
}
```

무조건 files 아래에만 생김

• API 17 이후로 고정 - 앱 전용으로 파일을 사용함을 지정
• 파일 내용을 추가하고자 할 경우 MODE_APPEND 사용

내부저장소 파일 사용 4

📁 파일 읽기

• File 클래스를 사용하여
파일 관리

• filesDir : 내부저장소
앱 전용 위치

```
val newFile = File ( filesDir, "file_name")
// val newFile = File( "${filesDir}/sub", "file_name")
// filesDir/sub/file_name 표현
```

~Reader나 ~Writer가 붙으면 String을 다룬다

```
// 기본 방법
val result = StringBuffer()

val fileReader = FileReader(newFile)
BufferedReader(fileReader).useLines {lines ->
    for (line in lines) {
        result.append(line+"\n")
    }
}
```

• useLines
: 파일의 모든 line 을
Sequence<String> 으로 전달
: 완료 후 close() 호출

```
// 추가 방법 - 앱에 지정되어 있는 전용 위치를 사용
context.openFileInput("file_name").bufferedReader().useLines { lines ->
    for (line in lines) {
        result.append(line+"\n")
    }
} filesDir에서만 읽어올 수 있음
```


내부저장소 파일 사용 5

이미지 파일 Bitmap 으로 읽어오기

```
val imageFile = File( filesDir, "image.jpg")
val bitmap = BitmapFactory.decodeFile(imageFile.path)
imageView.setImageBitmap(bitmap)
```

• 필요할 경우 읽어오기 관련 option 을 추가로 지정

이미지 파일 Bitmap 으로 저장하기

```
val imageFile = File( filesDir, "image.jpg")
val fos = FileOutputStream(imageFile)

// bitmap은 Bitmap 객체
bitmap.compress(Bitmap.CompressFormat.JPEG, 100, fos)
fos.close()
```

• 저장할 이미지 파일의 종류 지정
: JPEG, PNG 등

• 이미지 품질 지정
: 필요할 경우 값 변경

내부저장소 파일 사용 6

Glide 활용 이미지 파일 읽기

implementation 'com.github.bumptech.glide:glide:4.16.0' // build.gradle 추가

```
Glide.with(this@MainActivity)
    .load("${filesDir}/images/image.jpg")
    .into(binding.imageView)
```

• 기본위치/images/image.jpg

```
Glide.with(context)
    .load(imageUrl)
    .into(view)
```

Glide 활용 이미지 URL 읽기 & 파일 저장

```
Glide.with(this) // context
    .asBitmap()
    .load(url)
    .into(object: CustomTarget<Bitmap>( 350, 350 ) { // or Target.SIZE_ORIGINAL
        override fun onResourceReady( resource: Bitmap,
            transition: Transition<in Bitmap>?) {
            val imageFile = File ( "${filesDir}/images", "image.jpg")
            val fos = FileOutputStream(imageFile)
            resource.compress(Bitmap.CompressFormat.JPEG, 100, fos)
            fos.close()
        }
        override fun onLoadCleared(placeholder: Drawable?) {
            Log.d(TAG, "Image load cleared!")
        }
    })
```

• Bitmap 처리를 위해 지정

• 지정한 Target 이외의 객체로 결과를 처리

• Bitmap 준비
• 완료 시 호출

• 앱전용위치
/images/image.jpg

• Bitmap 준비 취소 시 호출

내부저장소 파일 사용 7

내부 저장소에 하위 디렉토리가 필요할 경우

```

fun createSubDirectory(baseDirectory: String, subDirectory: String) : Boolean {
    val directory = File(baseDirectory, subDirectory)
    if (!directory.exists()) {
        directory.mkdir()
        return true
    }
    return false
}
    
```

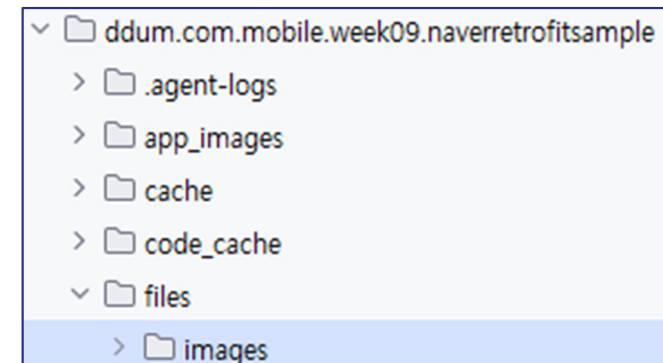
File

- 새로운 경로명에 해당하는 File 생성
- 디렉토리 생성
- 중첩 (base/sub/sub..) 디렉토리를 생성할 경우 mkdirs() 사용

◆파일 작업 전 실행 (앱 실행 시)

```

// 내부저장소 전용위치에 images 하위 디렉토리 생성
if ( !FileManager.createSubDirectory(filesDir, "images") ) {
    Log.d(TAG, "Sub directory error!")
}
    
```

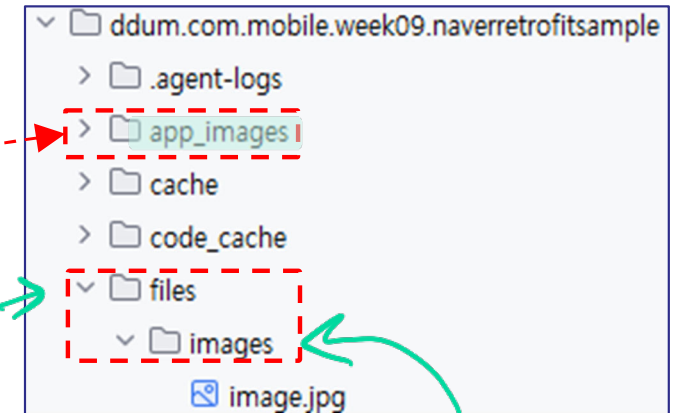


내부저장소 파일 사용 8

전용 디렉토리 생성하기

- ◆ 앱의 루트에 app_디렉토리명으로 생성

```
getDir("images", Context.MODE_PRIVATE)
```



파일 목록 보기

```
① var files : Array<String> = context.listFiles() // 앱 전용 위치
② val dir = File ( "dir_path" ) // dir_path 지정 위치
   files = dir.list()
```

파일 삭제

```
① context.deleteFile("file_name") // 앱 전용 위치 파일 삭제
② val deleteFile = File ("file_full_path") // 지정 위치 파일 삭제
   deleteFile.delete()
```

- ◆ 여러 파일 삭제 시 파일명을 바꿔가며 삭제 반복 수행

외부저장소의 파일 사용 1

SD카드 등 외부저장소에 앱 전용 또는 공유 목적으로 파일 사용 시 적용

외부저장소의 경우 접근이 보장되지 않으므로 사용 전 확인 필요

```
// Checks if a volume containing external storage is available
// for read and write.
fun isExternalStorageWritable(): Boolean {
    return Environment.getExternalStorageState() == Environment.MEDIA_MOUNTED
}

// Checks if a volume containing external storage is available to at least read.
fun isExternalStorageReadable(): Boolean {
    return Environment.getExternalStorageState() in
        arrayOf(Environment.MEDIA_MOUNTED, Environment.MEDIA_MOUNTED_READ_ONLY)
}
```

읽거나 쓰기 가능

출처: <https://developer.android.com/training/data-storage/app-specific#external-verify-availability>

외부저장소의 파일 사용 2

공용 위치에 저장할 경우

1. 앱전용 외부저장소 위치

```
val appSpecificExternalDir = File(getExternalFilesDir(null), "filename")
// null 대신 디렉토리 유형 지정 가능. 예: Environment.DIRECTORY_PICTURES
```

2. 모든 앱이 공통으로 접근할 수 있는 기본 디렉토리

- 예: Download(다운로드 파일), DCIM(사진), Movies(영상 파일), Music(음악 파일), Ringtones(벨소리) 등
- MediaStore API 또는 ACTION_CREATE_DOCUMENT 이용
- Android11 이후 MANAGE_EXTERNAL_STORAGE 권한 사용

.nomedia 파일

- ◆ MediaScanner 스캔 대상에서 제외하고자 할 때 사용
- ◆ .nomedia 파일(내용 없음)을 해당 디렉토리에 생성
 - 예) 특정 디렉토리의 사진들을 [갤러리] 앱에서 볼 수 없게 하고자 할 경우 해당 디렉토리에 .nomedia 파일 생성

파일 저장 시 파일명 지정 고려사항

기존 경로의 파일명을 사용

- 확장자가 jpg 또는 png 인지 확인 필요: contains()
- 없을 경우 추가

```
fun getImageFileName(path: String) : String {
    val fileName = path.slice(IntRange( path.lastIndexOf("/") + 1, path.length - 1))
    return fileName
}
```

https://shopping-phinf.pstatic.net/main_3248204/32482041666.20230725121007.jpg

◆ File.getName() 또는 Uri.getLastPathSegment() 사용 가능

시간 정보를 사용한 파일명 지정

◆ 현재시간을 사용하여 파일명 생성

```
fun getCurrentTime() : String {
    return SimpleDateFormat("yyyyMMdd_HH:mm:ss").format(Date())
}

Log.d(TAG, "file name: ${fileManager.getCurrentTime()}.jpg")
```

- 실제 파일의 포맷 확인 후 지정
- 예: 20000000_000000.jpg

SharedPreferences 의 사용 1

SharedPreferences 클래스

- ◆ Key-value 형태의 데이터를 영구 저장할 때 사용
- ◆ Context.getSharedPreferences(“파일명”, mode) 로 객체 획득
 - 여러 Activity 에서 공유할 SharedPreferences 가 필요할 때 사용
 - 파일명: preferences 값이 저장될 내부 파일 명
 - mode: 파일의 접근 모드, 디폴트 값 0 사용
 - Boolean, Float, Int, Long, String, String set 으로 저장 가능
- ◆ Context.getSharedPreferences()
 - 해당 Activity 전용의 SharedPreferences 가 필요할 때 사용

사용 시점

- ◆ SharedPreferences 읽기 → onCreate()
- ◆ SharedPreferences 쓰기 → onPause() 또는 특정 이벤트 시점 (저장 체크 등)

SharedPreferences 의 사용 2

읽기 및 쓰기

- ◆ *Datatype* : Int, Long, Boolean, String 등
- ◆ `putDatatype ("key_name", value)`
- ◆ `getDatatype ("key_name", [default_value])`

보통
onCreate
안에서

```
// SharedPreferences 객체 생성
val sharedPref = getPreferences(Context.MODE_PRIVATE)

// SharedPreferences 기록
val editor: SharedPreferences.Editor = sharedPref.edit()
with (editor) {
    putInt("value", 100)
    apply() // commit()
}
```

보통
onPause
안에서

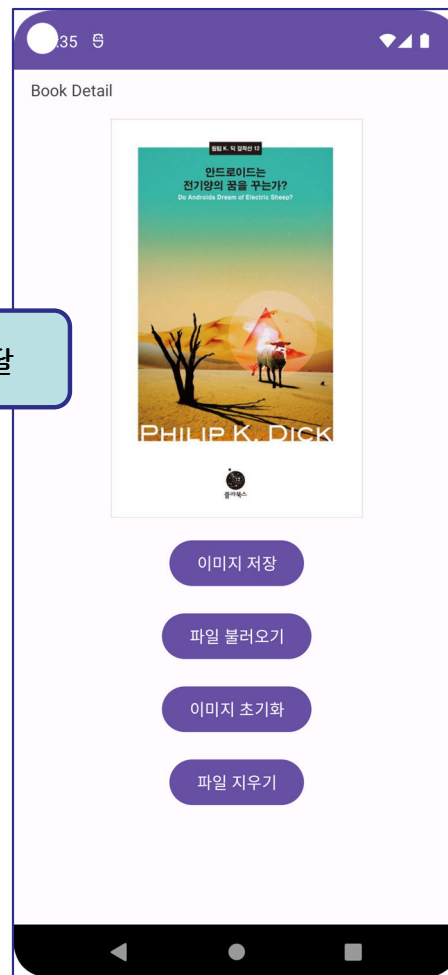
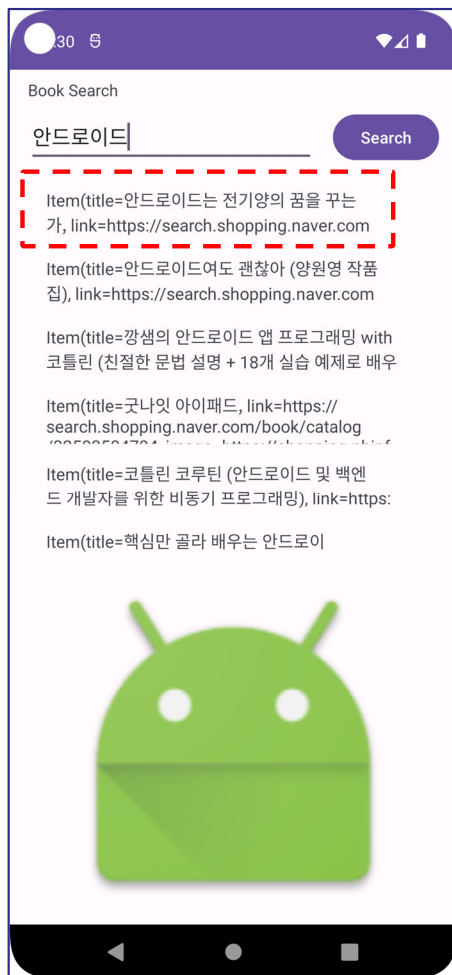
```
// SharedPreferences 읽기
val storedValue = sharedPref.getInt("value", 0)
```

0은 값이 없을 때 저장되는 디폴트 값

String일 경우 디폴트 벨류 없음

📱 Naver RetrofitSample_v2 프로젝트 사용

- ◆ Naver OpenAPI 사용 (ClientD, ClientSecret, 패키지명 설정할 것)
- ◆ 클릭한 항목의 이미지 url 을 intent 로 DetailActivity 에 전달
- ◆ DetailActivity 에서 url 의 Image 표시 (Glide 사용)



[이미지 저장 버튼]
- 내부저장소 지정위치/images 디렉토리에 "시간.jpg" 로 저장

[파일 불러오기 버튼]
- 저장한 파일을 불러와 ImageView 에 표시
(이미지 초기화 후 실행)

[이미지 초기화 버튼]
- ImageView 를 초기 이미지로 지정

[파일 지우기 버튼]
- 저장한 이미지 파일 삭제

참고

데이터 및 파일 저장소 개요

- ◆ <https://developer.android.com/training/data-storage?hl=ko>

외부저장소 확인

- ◆ <https://developer.android.com/training/data-storage/app-specific?hl=ko#external-verify-availability>

공용 저장소의 미디어 파일에 액세스

- ◆ <https://developer.android.com/training/data-storage/shared/media?hl=ko>

